

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Τμήμα Πληροφορικής & Τηλεπικοινωνιών

Μάθημα: Τεχνολογίες Εφαρμογών Διαδικτύου

Planent

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Ομάδα Εργασίας

Ζαχαράκης Αντώνιος — 1115202300047

Ιωάννης Οικονομίδης — 1115201600120

Ακαδημαϊκό Έτος 2025–2026

Εαρινό Εξάμηνο

Πίνακας Περιεχομένων

1. Εισαγωγή.....	4
2. Αρχιτεκτονική Συστήματος.....	5
2.1 Συνοπτική Επισκόπηση.....	5
2.2 Τεχνολογικό Στοιχείο (Tech Stack).....	5
2.3 Στρωματοποιημένη Αρχιτεκτονική Backend.....	6
3. Σχεδίαση Βάσης Δεδομένων.....	7
3.1 Κύριοι Πίνακες.....	7
3.2 Βασικές Σχέσεις και Περιορισμοί.....	7
3.3 Αρχικοποίηση.....	7
4. Backend — REST API.....	8
4.1 Δομή Πακέτων.....	8
4.2 Συνοπτικός Πίνακας Endpoints.....	8
4.3 Διαχείριση Σφαλμάτων.....	8
5. Frontend — React Εφαρμογή.....	9
5.1 Routing.....	9
5.2 Διαχείριση Κατάστασης.....	9
5.3 Επιλεγμένες Σχεδιαστικές Αποφάσεις.....	9
6. Λειτουργίες & Ρόλοι Χρηστών.....	10
6.1 Επισκέπτης (Guest).....	10
6.2 Συμμετέχων (Attendee).....	10
6.3 Διοργανωτής (Organizer).....	10
6.4 Διαχειριστής (Administrator).....	10
7. Ασφάλεια.....	11
7.1 Κρυπτογραφημένη Επικοινωνία (SSL/TLS).....	11
7.2 Αποθήκευση Κωδικών.....	11
7.3 JSON Web Tokens.....	11
7.4 Εξουσιοδότηση Βάσει Ρόλου.....	11
7.5 Προστασία από Spam / Σπατάλη Πόρων.....	11
8. Εξαγωγή Εκδηλώσεων σε XML / JSON.....	12
8.1 Συμβατότητα με το DTD.....	12
8.2 JSON Output.....	12
8.3 Παράδειγμα Χρήσης.....	12
9. Σύστημα Συστάσεων (Biased Matrix Factorization).....	13
9.1 Σχεδιαστική Προσέγγιση.....	13
9.2 Σημεία προς Ολοκλήρωση.....	13
9.3 Σημείωση.....	13

10. Οδηγίες Εγκατάστασης & Εκτέλεσης.....	14
10.1 Προαπαιτούμενα.....	14
10.2 Backend.....	14
10.3 Frontend.....	14
10.4 Εναλλακτικά: Docker.....	14
11. Επίλογος.....	15

1. Εισαγωγή

Η παρούσα εργασία υλοποιήθηκε στο πλαίσιο του μαθήματος «Τεχνολογίες Εφαρμογών Διαδικτύου» και είχε ως στόχο τη σχεδίαση και ανάπτυξη μιας ολοκληρωμένης διαδικτυακής εφαρμογής διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων θέσεων, με την κωδική ονομασία Planent.

Η εφαρμογή ακολουθεί την κλασική αρχιτεκτονική web browser / web server: ο χρήστης αλληλεπιδρά με την υπηρεσία αποκλειστικά μέσω του φυλλομετρητή, ενώ η λογική και τα δεδομένα διαχειρίζονται από ένα Spring Boot νωτιαίο άκρο (backend) που εκθέτει REST API πάνω από SSL/TLS. Όλη η επικοινωνία πελάτη-εξυπηρετητή πιστοποιείται με χρήση JSON Web Tokens (JWT) και η εξουσιοδότηση γίνεται βάσει του ρόλου του χρήστη: Διαχειριστής, Διοργανωτής, Συμμετέχων ή Επισκέπτης.

Στα επόμενα κεφάλαια παρουσιάζονται με συντομία:

- Η συνολική αρχιτεκτονική του συστήματος και οι τεχνολογίες που χρησιμοποιήθηκαν (Κεφάλαιο 2).
- Η σχεδίαση της σχεσιακής βάσης δεδομένων (Κεφάλαιο 3).
- Η οργάνωση του backend και τα κυριότερα REST endpoints (Κεφάλαιο 4).
- Η δομή και τα κυριότερα στοιχεία του frontend (Κεφάλαιο 5).
- Οι λειτουργίες που είναι διαθέσιμες ανά ρόλο χρήστη (Κεφάλαιο 6).
- Οι μηχανισμοί ασφάλειας που εφαρμόστηκαν (Κεφάλαιο 7).
- Η εξαγωγή εκδηλώσεων σε XML/JSON σύμφωνα με το ζητούμενο DTD (Κεφάλαιο 8).
- Η σχεδίαση του αλγορίθμου συστάσεων (Κεφάλαιο 9).
- Οδηγίες εγκατάστασης και εκτέλεσης (Κεφάλαιο 10) και επίλογος (Κεφάλαιο 11).

2. Αρχιτεκτονική Συστήματος

Το σύστημα είναι οργανωμένο σε δύο ανεξάρτητα deployable τμήματα, το backend και το frontend, τα οποία επικοινωνούν αποκλειστικά μέσω HTTPS και ενός REST API.

2.1 Συνοπτική Επισκόπηση

Το frontend είναι μια Single-Page Application (SPA) γραμμένη σε React με TypeScript, η οποία εξυπηρετείται από Vite κατά την ανάπτυξη και ως στατικά αρχεία σε production. Επικοινωνεί με το backend μέσω HTTPS αιτημάτων στα οποία προσθέτει αυτόματα το JWT του τρέχοντος χρήστη ως Bearer token.

Το backend είναι μια Spring Boot 4 εφαρμογή σε Java 21 που:

- Παρέχει το REST API της εφαρμογής (controllers).
- Διαχειρίζεται τα δεδομένα μέσω Spring Data JPA / Hibernate, απεικονίζοντας το σχεσιακό μοντέλο της MySQL σε αντικειμενοστραφές μοντέλο.
- Υλοποιεί τον μηχανισμό αυθεντικοποίησης (JWT) και εξουσιοδότησης (role-based authorization) μέσω του Spring Security.
- Εκτελεί ασύγχρονη αποστολή μηνυμάτων προς τους συμμετέχοντες όταν ακυρώνεται μία εκδήλωση, μέσω του μηχανισμού των Application Events.

2.2 Τεχνολογικό Στοιχείο (Tech Stack)

Επίπεδο	Τεχνολογίες
Frontend	React 19, TypeScript, Vite, React Router, Axios, React-Leaflet (OpenStreetMap)
Backend	Java 21, Spring Boot 4, Spring Web MVC, Spring Security, Spring Data JPA, Jackson (JSON & XML)
Database	MySQL 8 (σχεσιακή ΒΔ, MySQL Connector/J)
Ασφάλεια	SSL/TLS (PKCS12 keystore), JWT (jjwt), BCrypt password hashing, CORS, Stateless Sessions
Build	Maven Wrapper (backend), npm + Vite (frontend), Dockerfiles για ανάπτυξη και παραγωγή

2.3 Στρωματοποιημένη Αρχιτεκτονική Backend

Ακολουθήθηκε η κλασική στρωματοποίηση Spring εφαρμογών, με σαφή διαχωρισμό αρμοδιοτήτων:

- **Controllers:** δέχονται τα HTTP αιτήματα, εκτελούν validation στα DTOs και αναθέτουν τη λογική στα services.
- **Services:** ενσωματώνουν την επιχειρησιακή λογική, διαχειρίζονται transactions και επικοινωνούν με τα repositories.
- **Repositories:** Spring Data JPA interfaces, επεκτάσιμα με JpaSpecificationExecutor για δυναμικά queries (αναζήτηση εκδηλώσεων με πολλαπλά κριτήρια).
- **Models:** JPA entities με relationships, validation rules και domain-level invariants (π.χ. έλεγχος χωρητικότητας πριν την αποθήκευση).

- **DTOs / Mappers:** ξεχωριστοί τύποι για request/response, ώστε να μην εκτίθενται τα entities και να είναι σταθερό το API contract.
- **Specifications:** δυναμικά JPA criteria predicates για την υλοποίηση φίλτρων αναζήτησης.
- **Listeners:** Transactional Event Listeners για ασύγχρονες ενέργειες (π.χ. μαζική αποστολή μηνυμάτων ακύρωσης).

3. Σχεδίαση Βάσης Δεδομένων

Η βάση δεδομένων σχεδιάστηκε ώστε να καλύπτει πλήρως το ζητούμενο DTD και τις απαιτήσεις λειτουργικότητας. Χρησιμοποιείται MySQL ως RDBMS και το Hibernate ως ORM. Στη φάση παραγωγής η ιδιότητα `ddl-auto` είναι ορισμένη σε `validate`, ώστε το backend να επικυρώνει τη συμβατότητα μεταξύ entities και πραγματικού σχήματος.

3.1 Κύριοι Πίνακες

Πίνακας	Περιγραφή
User	Στοιχεία χρήστη (username, password hash, στοιχεία επικοινωνίας, διεύθυνση, γεωγραφική θέση, ΑΦΜ, ρόλος admin, κατάσταση έγκρισης).
Event	Εκδηλώσεις (τίτλος, τύπος, χώρος, διεύθυνση/γεωγραφικές συντεταγμένες, χρόνοι έναρξης/λήξης, χωρητικότητα, περιγραφή, κατάσταση: DRAFT / PUBLISHED / COMPLETED / CANCELLED, organizer).
Category	Λίστα κατηγοριών (Music, Sports, Conference, κ.λπ.).
Event_Category	Πίνακας ένωσης Event ↔ Category (πολλαπλές κατηγορίες ανά εκδήλωση).
Event_TicketType	Τύποι εισιτηρίων για κάθε εκδήλωση (όνομα, τιμή, ποσότητα, διαθέσιμα).
Event_Media	Προαιρετικές φωτογραφίες/οπτικό υλικό ανά εκδήλωση.
Booking	Κρατήσεις (attendee, ticket_type, πλήθος εισιτηρίων, συνολικό κόστος, χρονοσήμανση, κατάσταση: PENDING / CONFIRMED / CANCELLED).
Message	Μηνύματα μεταξύ χρηστών (sender, receiver, event, σώμα, isRead, σημαίες λογικής διαγραφής deletedBySender/deletedByReceiver).

3.2 Βασικές Σχέσεις και Περιορισμοί

- Κάθε εκδήλωση συνδέεται με ακριβώς έναν διοργανωτή (User) και μπορεί να ανήκει σε πολλές κατηγορίες (many-to-many).
- Κάθε εκδήλωση έχει έναν ή περισσότερους τύπους εισιτηρίων· το άθροισμα των ποσοτήτων ελέγχεται από την επιχειρησιακή λογική (Event.validateTicketCapacity) ώστε να μην υπερβαίνει τη συνολική χωρητικότητα.
- Κάθε κράτηση δείχνει σε έναν συγκεκριμένο τύπο εισιτηρίου, ο οποίος διαθέτει πεδίο available που μειώνεται ατομικά με κάθε επιτυχημένη κράτηση.
- Στη διαγραφή χρήστη ή εκδήλωσης ορίζονται αντίστοιχα cascade rules μέσω JPA (CascadeType.ALL + orphanRemoval) για τα media, τις κατηγορίες και τους τύπους εισιτηρίων.
- Τα μηνύματα δεν διαγράφονται φυσικά· χρησιμοποιούνται οι σημαίες deletedBySender/deletedByReceiver για λογική διαγραφή ανά χρήστη, διατηρώντας την ιστορικότητα.

3.3 Αρχικοποίηση

Κατά την πρώτη εκκίνηση του backend (PlanentApplication.initDatabase) δημιουργείται αυτόματα ένας ενσωματωμένος διαχειριστής με username admin (αν δεν υπάρχει ήδη). Ο κωδικός αποθηκεύεται με BCrypt· οι λοιπές πληροφορίες (διεύθυνση, τηλέφωνο κ.λπ.) είναι placeholder και μπορούν να τροποποιηθούν.

4. Backend — REST API

Το backend εκθέτει ένα RESTful API που καταναλώνεται αποκλειστικά από το frontend. Όλα τα endpoints απαιτούν HTTPS και — εκτός από τα δημόσια — έγκυρο JWT στο Authorization header.

4.1 Δομή Πακέτων

Όλα τα Java πακέτα βρίσκονται κάτω από `com.uoa.planent` και είναι οργανωμένα ως εξής: `annotation`, `config` (Security/Authentication/Jackson), `controller`, `dto` (`booking/event/event.export/message/user`), `event` (custom application events), `exception`, `listener`, `mapper`, `model`, `repository`, `security`, `service`, `specification`.

4.2 Συνοπτικός Πίνακας Endpoints

Method	Endpoint	Περιγραφή
POST	/auth/register	Εγγραφή νέου χρήστη (PENDING).
POST	/auth/login	Σύνδεση και έκδοση JWT.
GET	/users/me	Τα στοιχεία του τρέχοντος χρήστη.
GET	/users	Λίστα όλων των χρηστών (admin only).
PATCH	/users/{id}/approve	Έγκριση αίτησης εγγραφής (admin only).
DELETE	/users/{id}	Απόρριψη/διαγραφή χρήστη (admin only).
GET	/events/search	Σελιδοποιημένη αναζήτηση εκδηλώσεων με πολλαπλά φίλτρα.
POST	/events	Δημιουργία εκδήλωσης από διοργανωτή.
PATCH	/events/{id}	Ενημέρωση / δημοσίευση / ακύρωση εκδήλωσης.
DELETE	/events/{id}	Διαγραφή εκδήλωσης (μόνο πριν τη δημοσίευση/πρώτη κράτηση).
POST	/bookings	Δημιουργία κράτησης (atomic έλεγχος και μείωση διαθέσιμων θέσεων).
GET/ POST/ DELETE	/messages, /messages/inbox, /messages/sent, /messages/{id}, /messages/unread-count	Διαχείριση μηνυμάτων μεταξύ διοργανωτή ↔ συμμετεχόντων.
GET	/admin/export/events?format=xml json	Εξαγωγή όλων των εκδηλώσεων σε XML ή JSON (admin only).

4.3 Διαχείριση Σφαλμάτων

Όλα τα σφάλματα κανάλιζονται μέσα από έναν `GlobalExceptionHandler` (`@RestControllerAdvice`), ο οποίος μετατρέπει τα domain exceptions (`ValidationException`, `IllegalStateException`, `ResourceNotFoundException`, `AccessDeniedException`, `BadCredentialsException`, `DisabledException`, `DataIntegrityViolationException`, `MethodArgumentNotValidException` κ.ά.) σε τυποποιημένα RFC 7807 ProblemDetail JSON responses με κατάλληλο HTTP status.

5. Frontend — React Εφαρμογή

Το frontend είναι μια Single-Page Application που υλοποιήθηκε σε React 19 με TypeScript. Η οργάνωση του κώδικα είναι λειτουργικά διαχωρισμένη σε api, components, context, pages, styles και types.

5.1 Routing

Η πλοήγηση γίνεται με React Router (BrowserRouter). Οι κυριότερες διαδρομές είναι:

- **/** : Σελίδα καλωσορίσματος (WelcomePage) με προβολή ενδεικτικών εκδηλώσεων.
- **/login, /register, /pending-approval** : Σελίδες αυθεντικοποίησης και αναμονής έγκρισης.
- **/events, /events/:id** : Πλοήγηση/αναζήτηση εκδηλώσεων και σελίδα λεπτομερειών εκδήλωσης με χάρτη OpenStreetMap.
- **/create-event, /edit-event/:id** : Δημιουργία και επεξεργασία εκδηλώσεων (Διοργανωτής).
- **/my-events, /my-events/:id/bookings, /my-events/:id/broadcast** : Διαχείριση εκδηλώσεων και κρατήσεων από τον διοργανωτή.
- **/my-bookings, /book/:id** : Κρατήσεις του Συμμετέχοντα.
- **/messages, /messages/new, /messages/compose** : Σύστημα μηνυμάτων.
- **/profile, /admin** : Προφίλ χρήστη και διαχειριστικός πίνακας.

5.2 Διαχείριση Κατάστασης

Η καθολική κατάσταση (συνδεδεμένος χρήστης, JWT, theme) διαχειρίζεται μέσω React Context (AuthContext, ThemeContext). Το JWT αποθηκεύεται στο localStorage και προσαρτάται αυτόματα σε κάθε αίτημα μέσω ενός Axios interceptor (api/axiosConfig.ts).

5.3 Επιλεγμένες Σχεδιαστικές Αποφάσεις

- Στη σελίδα κρατήσεων υπάρχει επιβεβαιωτικό dialog πριν την οριστική υποβολή, καθώς από τις απαιτήσεις η κράτηση δεν αναιρείται.
- Στη σελίδα λεπτομερειών εκδήλωσης ενσωματώνεται διαδραστικός χάρτης OpenStreetMap με Leaflet, που εμφανίζει την ακριβή τοποθεσία διεξαγωγής.
- Στην εγγραφή και δημιουργία εκδήλωσης χρησιμοποιείται LocationAutocomplete component το οποίο επικοινωνεί με υπηρεσία γεωκωδικοποίησης για αυτόματη συμπλήρωση διεύθυνσης, πόλης, χώρας και συντεταγμένων.
- Το unread badge των μηνυμάτων ενημερώνεται αυτόματα σε κάθε αλλαγή route, μέσω polling του /messages/unread-count.
- Όλα τα paginated views (events, my-events, bookings, messages) χρησιμοποιούν ένα κοινό Pagination component.
- Υποστηρίζονται light/dark themes μέσω CSS variables και του ThemeContext.

6. Λειτουργίες & Ρόλοι Χρηστών

Η εφαρμογή υποστηρίζει τέσσερις ρόλους, ο καθένας με διακριτές δυνατότητες.

6.1 Επισκέπτης (Guest)

- Πρόσβαση στη σελίδα καλωσορίσματος και προβολή ενδεικτικών εκδηλώσεων.
- Πλοήγηση και αναζήτηση όλων των δημοσιευμένων εκδηλώσεων (φίλτρα: κατηγορία, πόλη, μέγιστη τιμή, ημερομηνίες, ελεύθερο κείμενο σε τίτλο/περιγραφή).
- Προβολή πλήρων λεπτομερειών εκδήλωσης (συμπεριλαμβανομένου χάρτη).
- Δεν επιτρέπεται η πραγματοποίηση κρατήσεων ούτε η αποστολή μηνυμάτων.

6.2 Συμμετέχων (Attendee)

- Όλες οι δυνατότητες του Επισκέπτη.
- Πραγματοποίηση κρατήσεων σε δημοσιευμένες εκδηλώσεις με επιβεβαίωση πριν την υποβολή.
- Προβολή προσωπικών κρατήσεων (/my-bookings) και ακύρωση κράτησης (αν η εκδήλωση δεν έχει ολοκληρωθεί).
- Αμφίδρομη επικοινωνία μέσω μηνυμάτων με τον διοργανωτή εκδήλωσης για την οποία έχει υποβάλει κράτηση.

6.3 Διοργανωτής (Organizer)

Κάθε εγγεγραμμένος χρήστης μπορεί να λειτουργεί ως διοργανωτής δημιουργώντας εκδηλώσεις. Επιπλέον των δικαιωμάτων του Συμμετέχοντα έχει:

- Δημιουργία εκδήλωσης με όλα τα πεδία του DTD (τίτλος, τύπος, χώρος, διεύθυνση/γεωγραφικές συντεταγμένες, χρόνοι, χωρητικότητα, κατηγορίες, τύποι εισιτηρίων, φωτογραφίες).
- Δημοσίευση (DRAFT → PUBLISHED) και ακύρωση (PUBLISHED → CANCELLED) εκδηλώσεων.
- Διαγραφή εκδήλωσης μόνο εφόσον είναι DRAFT και χωρίς κρατήσεις.
- Προβολή των κρατήσεων που έχουν γίνει για κάθε εκδήλωσή του.
- Αποστολή ενημερωτικού μηνύματος προς όλους τους συμμετέχοντες (broadcast). Σε ακύρωση εκδήλωσης το μήνυμα αποστέλλεται αυτόματα από τον listener του συστήματος.

6.4 Διαχειριστής (Administrator)

- Πρόσβαση σε ξεχωριστή σελίδα διαχείρισης (/admin) με tabs Users και Events.
- Πλοήγηση και προβολή λεπτομερειών όλων των χρηστών.
- Έγκριση ή απόρριψη εκκρεμών εγγραφών νέων χρηστών.
- Δημοσίευση ή ακύρωση εκδηλώσεων χρηστών.
- Εξαγωγή όλων των εκδηλώσεων σε XML ή JSON (βλ. Κεφάλαιο 8).

7. Ασφάλεια

Η ασφάλεια αντιμετωπίστηκε σε όλα τα επίπεδα της εφαρμογής:

7.1 Κρυπτογραφημένη Επικοινωνία (SSL/TLS)

Το backend εκκινεί πάνω από HTTPS με χρήση PKCS12 keystore (server.ssl.enabled=true στο application.properties). Αντίστοιχα, το frontend dev server (Vite) χρησιμοποιεί το plugin @vitejs/plugin-basic-ssl ώστε όλη η ανάπτυξη να γίνεται μέσω HTTPS από την αρχή.

7.2 Αποθήκευση Κωδικών

Όλοι οι κωδικοί χρηστών αποθηκεύονται hashed με BCrypt (PasswordEncoder bean στο AuthenticationConfig). Δεν εκτίθεται ποτέ ο κωδικός σε κανένα DTO ή response.

7.3 JSON Web Tokens

Η αυθεντικοποίηση είναι stateless. Κατά τη σύνδεση εκδίδεται JWT (HS256) με ορισμένη διάρκεια ζωής (application.security.jwt-ttl, 1 ώρα by default). Ένα JwtAuthFilter παρεμβάλλεται πριν τον UsernamePasswordAuthenticationFilter και:

- Διαβάζει το Authorization: Bearer <token> header.
- Επαληθεύει την υπογραφή και τη λήξη του token.
- Φορτώνει τον UserDetails μέσω του UserDetailsServiceImpl και ορίζει το SecurityContext για το συγκεκριμένο request.

7.4 Εξουσιοδότηση Βάσει Ρόλου

Η εξουσιοδότηση γίνεται με μέθοδο-επίπεδες ανοτατζίες (@PreAuthorize) και declarative rules στο SecurityFilterChain. Παραδείγματα:

- @PreAuthorize("hasAuthority('ADMIN')") σε όλα τα endpoints διαχείρισης χρηστών και εξαγωγής.
- Custom expressions όπως @userService.isUserOwnerOrAdmin για να επιτρέπεται η πρόσβαση μόνο στον ιδιοκτήτη του πόρου ή στον admin.
- Τα /auth/** είναι permitAll(), τα GET /events/** είναι permitAll() για τους Επισκέπτες, ενώ όλα τα υπόλοιπα απαιτούν authenticated().

7.5 Προστασία από Spam / Σπατάλη Πόρων

- CORS: αυστηρά configured να δέχεται μόνο το URL του frontend (frontend.url property).
- Stateless sessions (SessionCreationPolicy.STATELESS) — δεν δημιουργούνται HTTP sessions, οπότε αποτρέπονται επιθέσεις session fixation.
- Validation στα DTOs με Jakarta Bean Validation (@NotBlank, @Pattern, @Email, @Future, @Min κ.λπ.) που εφαρμόζεται πριν την επιχειρησιακή λογική.

8. Εξαγωγή Εκδηλώσεων σε XML / JSON

Ο διαχειριστής μπορεί να εξάγει όλες τις εκδηλώσεις του συστήματος μέσω του endpoint `/admin/export/events?format=xml|json`. Η υλοποίηση χρησιμοποιεί ξεχωριστά Export DTOs (πακέτο `dto.event.export`) και Jackson `XmlMapper` / `JsonMapper`.

8.1 Συμβατότητα με το DTD

Τα DTOs σχεδιάστηκαν ώστε η παραγόμενη XML να ταυτίζεται ακριβώς με το DTD της εκφώνησης:

- `@JsonPropertyOrder` για να διατηρείται η σωστή σειρά στοιχείων (Title, Category+, EventType, Venue, ...).
- `@JacksonXmlProperty(localName=...)` για ονόματα στοιχείων με κεφαλαία (Title, Venue, Category κ.λπ.).
- `@JacksonXmlProperty(isAttribute=true)` για τα attributes όπως EventID, TicketTypeID, BookingID, UserID, Latitude, Longitude.
- `@JacksonXmlElementWrapper` για τα wrapper elements (TicketTypes, Bookings, Media). Το Category παράγεται χωρίς wrapper, ως επαναλαμβανόμενο στοιχείο μέσα στο Event.
- `@JsonInclude(NON_NULL)` στο GeoLocation και `@JsonInclude(NON_EMPTY)` στο Media, σύμφωνα με το «?» του DTD (προαιρετικά στοιχεία).

8.2 JSON Output

Η JSON εκδοχή παράγεται από τα ίδια DTOs με τη χρήση του default `JsonMapper` του Spring Boot και `writerWithDefaultPrettyPrinter`, για ευανάγνωστη μορφή.

8.3 Παράδειγμα Χρήσης

```
GET https://localhost:5000/api/admin/export/events?format=xml
Authorization: Bearer <admin_jwt>
```

Η απάντηση επιστρέφεται ως attachment (Content-Disposition: attachment; filename="events-export.xml") και το frontend (AdminPage) εξάγει το αρχείο μέσω download link στο browser.

9. Σύστημα Συστάσεων (Biased Matrix Factorization)

Η εργασία περιλαμβάνει την υλοποίηση αλγορίθμου συστάσεων Biased Matrix Factorization από το μηδέν. Το συγκεκριμένο τμήμα βρίσκεται σε εξέλιξη και θα παραδοθεί με την τελική έκδοση της εργασίας.

9.1 Σχεδιαστική Προσέγγιση

Ο αλγόριθμος θα εκπαιδευτεί στο dataset που παρέχεται από το eclass και θα προσφέρει εξατομικευμένες προτάσεις εκδηλώσεων σε κάθε χρήστη βάσει του ιστορικού κρατήσεων και των εκδηλώσεων που έχει επισκεφθεί. Αν ο χρήστης δεν διαθέτει ιστορικό κρατήσεων, ο αλγόριθμος θα λειτουργεί αποκλειστικά βάσει των views εκδηλώσεων.

9.2 Σημεία προς Ολοκλήρωση

- Καταγραφή των επισκέψεων (views) χρήστη σε εκδηλώσεις σε κατάλληλο πίνακα της βάσης.
- Υλοποίηση του αλγορίθμου Biased Matrix Factorization (παράγοντες χρήστη, παράγοντες αντικειμένου, global mean και user/item biases) από το μηδέν, με stochastic gradient descent.
- Εκπαίδευση και αποθήκευση των παραμέτρων (latent factors) στη ΒΔ ή σε αρχείο, με δυνατότητα περιοδικής επανεκπαίδευσης.
- Νέο REST endpoint (π.χ. GET /events/recommended) που θα επιστρέφει top-K προτεινόμενες εκδηλώσεις για τον τρέχοντα χρήστη.
- Ενσωμάτωση στο frontend σε κατάλληλο τμήμα της διεπαφής (π.χ. προτεινόμενες εκδηλώσεις στη σελίδα καλωσορίσματος ή στην αρχική των εγγεγραμμένων χρηστών).

9.3 Σημείωση

Στην τελική έκδοση του παρόντος εγγράφου το κεφάλαιο θα συμπληρωθεί με αναλυτική περιγραφή της μαθηματικής διατύπωσης, της διαδικασίας εκπαίδευσης (hyperparameters, learning rate, λ , epochs), των μετρικών αξιολόγησης (RMSE/MAE σε hold-out set) και των αποτελεσμάτων.

10. Οδηγίες Εγκατάστασης & Εκτέλεσης

10.1 Προαπαιτούμενα

- Java 21 (JDK)
- Maven 3.9+ (διατίθεται και το Maven Wrapper mvnw)
- Node.js 20+ και npm 10+
- MySQL Server 8+ (με δημιουργημένη βάση π.χ. planent)
- Σύγχρονος browser με υποστήριξη ES2022 (Chrome, Firefox, Edge)

10.2 Backend

Στον φάκελο backend δημιουργείται αρχείο .env (βλ. .env.example) με τις παρακάτω μεταβλητές περιβάλλοντος:

```
DB_URL=jdbc:mysql://localhost:3306/planent
DB_USERNAME=<mysql_user>
DB_PASSWORD=<mysql_password>
JWT_SECRET_KEY=<κλειδί τουλάχιστον 32 χαρακτήρων>
SSL_KEYSTORE_PASSWORD=<password του keystore.p12>
```

Στη συνέχεια εκκινούμε το backend (από τον φάκελο backend):

```
./mvnw spring-boot:run
```

Το backend ακούει σε <https://localhost:5000/api>. Στην πρώτη εκκίνηση δημιουργείται αυτόματα ο χρήστης admin / admin.

10.3 Frontend

Στον φάκελο frontend ορίζουμε το base URL του API σε αρχείο .env:

```
VITE_API_BASE_URL=https://localhost:5000/api
```

Εγκαθιστούμε τις εξαρτήσεις και εκκινούμε σε development mode:

```
npm install
npm run dev
```

Το frontend είναι διαθέσιμο στο <https://localhost:5173>. Η πρώτη επίσκεψη απαιτεί αποδοχή του self-signed πιστοποιητικού (τόσο για το frontend όσο και για το backend).

10.4 Εναλλακτικά: Docker

Παρέχονται Dockerfile και Dockerfile.dev για το frontend, ώστε να μπορεί να εκτελεστεί σε container χωρίς τοπική εγκατάσταση Node.js. Αντίστοιχα, το backend μπορεί να εκτελεστεί ως `spring-boot:run jar` ή μέσω κατάλληλου Dockerfile που μπορεί να προστεθεί στο μέλλον.

11. Επίλογος

Η ανάπτυξη της εφαρμογής Planent αποτέλεσε μια ολοκληρωμένη άσκηση σχεδίασης και υλοποίησης σύγχρονης διαδικτυακής εφαρμογής, με χρήση τεχνολογιών που συναντώνται και στην επαγγελματική πρακτική: Spring Boot με JPA στο backend, React με TypeScript στο frontend, JWT-based authentication, RESTful design, και HTTPS σε όλη τη ροή.

Κατά τη διάρκεια της υλοποίησης αντιμετωπίστηκαν αρκετές προκλήσεις:

- Η διασφάλιση των αναλλοίωτων (invariants) γύρω από τη χωρητικότητα και τη διαθεσιμότητα εισιτηρίων — λύθηκε με μετατόπιση της λογικής μέσα στο domain model (`Event.validate`, `EventTicketType.tryBook`) και transactional methods στο service layer.
- Η σωστή σειρά και ονοματολογία των στοιχείων κατά την εξαγωγή XML ώστε να συμμορφώνεται με το DTD — λύθηκε με προσεκτική χρήση των annotations `@JsonPropertyOrder`, `@JacksonXmlProperty` και `@JacksonXmlElementWrapper`.
- Ο συνδυασμός self-signed SSL πιστοποιητικών σε frontend και backend κατά την ανάπτυξη — απαιτήσε ρητή αποδοχή και των δύο πιστοποιητικών από τον browser.
- Η ασύγχρονη αποστολή μαζικών μηνυμάτων στους συμμετέχοντες κατά την ακύρωση εκδήλωσης χωρίς να καθυστερεί την απάντηση του API — λύθηκε με Spring Application Events και `@TransactionalEventListener` στο φάση `AFTER_COMMIT`.

Στα επόμενα βήματα της εργασίας θα ολοκληρωθεί και θα ενσωματωθεί ο αλγόριθμος συστάσεων Biased Matrix Factorization, ο οποίος θα προσφέρει εξατομικευμένες προτάσεις στους χρήστες με βάση το ιστορικό αλληλεπιδράσεών τους με την πλατφόρμα.